

RAG 系统部署文档

基于 DeepSeek R1 + Ollama + LangChain + Streamlit

生成日期: 2026-07-15

部署环境: Windows 11 / Python 3.11

1. 环境准备

本机存在多个 Python 解释器 (hermes venv / Local Program / Roaming), 导致 pip install 与 streamlit 使用不同 Python。最终统一使用 Local Program Python 安装所有包。

1.1 安装的 pip 包

```
#
streamlit
langchain==1.2.10      #  prompts/chains
langchain-community
langchain-experimental
langchain-classic      #  langchain
langchain-core         #  PromptTemplate

# PDF
pdfplumber

#
faiss-cpu

# Embedding +
sentence-transformers
torchvision            #  transformers
torch                  #  torchvision  2.13.0+cpu

# PDF
fpdf2
```

1.2 安装命令

```
set HF_ENDPOINT=https://hf-mirror.com
pip install streamlit langchain==1.2.10 langchain-community langchain-experimental \
    langchain-classic langchain-core pdfplumber faiss-cpu sentence-transformers
pip install torchvision --index-url https://download.pytorch.org/whl/cpu
```

注：中国网络环境下需设置 HF_ENDPOINT 镜像。torchvision 从 PyTorch 官方 CPU index 安装以避免 CUDA 依赖。

2. 模型与工具

组件	工具	模型
LLM	Ollama (本地服务)	deepseek-r1:7b
Embedding	HuggingFace (sentence-transformers)	all-mpnet-base-v2
文档加载	PDFPlumberLoader	—
向量库	FAISS	—
文本分块	SemanticChunker	—
前端	Streamlit	—

2.1 Ollama 模型列表

```
ollama list
deepseek-r1:7b      4.7 GB
gemma2:2b          1.6 GB
qwen2:7b           4.4 GB
deepseek-coder:6.7b 3.8 GB
```

2.2 Embedding 配置

Embedding 模型 all-mpnet-base-v2 从 HuggingFace 镜像站 (hf-mirror.com) 下载。首次启动约下载 1GB，后续有本地缓存。

```
import os
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
```

3. 完整代码

文件路径: ai/Main.py

```
import streamlit as st
import tempfile
import os

os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"

from langchain_community.document_loaders import PDFPlumberLoader
from langchain_experimental.text_splitter import SemanticChunker
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_community.llms import Ollama
from langchain_core.prompts import PromptTemplate
from langchain_classic.chains.llm import LLMChain
from langchain_classic.chains.combine_documents.stuff import StuffDocumentsChain
from langchain_classic.chains import RetrievalQA

# Theme colors
primary_color = "#1E90FF"
secondary_color = "#FF6347"
background_color = "#F5F5F5"
text_color = "#4561e9"

st.markdown(f"""<style>...""", unsafe_allow_html=True)
st.title(" DeepSeek R1 Ollama RAG ")

uploaded_file = st.file_uploader(" PDF ", type="pdf")

if uploaded_file:
    docs = documents = retriever = llm = None
    try:
        with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp:
            tmp.write(uploaded_file.getvalue())
            tmp_path = tmp.name
        loader = PDFPlumberLoader(tmp_path)
        docs = loader.load()
    except Exception as e: st.error(f"PDF: {e}")
    finally:
        if os.path.exists(tmp_path): os.unlink(tmp_path)
    if docs is None: st.stop()

    try:
        text_splitter = SemanticChunker(
            HuggingFaceEmbeddings(model_name="all-mpnet-base-v2"))
        documents = text_splitter.split_documents(docs)
    except Exception as e: st.error(f": {e}")
    if documents is None: st.stop()

    try:
        embedder = HuggingFaceEmbeddings(model_name="all-mpnet-base-v2")
        vector = FAISS.from_documents(documents, embedder)
        retriever = vector.as_retriever(search_type="similarity", k=3)
    except Exception as e: st.error(f": {e}")
    if retriever is None: st.stop()

    try:
        llm = Ollama(model="deepseek-r1:7b")
    except Exception as e: st.error(f"Ollama: {e}")
    if llm is None: st.stop()

    prompt = ""
```

1. Use context below.
2. If unknown say "I don't know".
3. Keep answer 3-4 sentences.

Context: {context}

Question: {question}

"""

```
QA_CHAIN_PROMPT = PromptTemplate.from_template(prompt)
llm_chain = LLMChain(llm=llm, prompt=QA_CHAIN_PROMPT, verbose=True)
document_prompt = PromptTemplate(
    input_variables=["page_content", "source"],
    template="Content: {page_content}\nSource: {source}")
combine_documents_chain = StuffDocumentsChain(
    llm_chain=llm_chain,
    document_variable_name="context",
    document_prompt=document_prompt)
qa = RetrievalQA(
    combine_documents_chain=combine_documents_chain,
    verbose=True, retriever=retriever,
    return_source_documents=True)
```

```
user_input = st.text_input(" PDF ")
if user_input:
    with st.spinner("..."):
        try:
            response = qa(user_input)["result"]
            st.write("", response)
        except Exception as e: st.error(f": {e}")
```

else:

```
st.write(" PDF ")
```

4. 使用步骤

4.1 启动 Ollama

```
ollama serve
ollama pull deepseek-r1:7b    #
```

4.2 启动应用

```
#
cd D:\yangzhi\com.yz.python

# streamlit python
streamlit run ai/Main.py

# Python
"C:\Users\yangz\AppData\Local\Programs\Python\Python311\python.exe" -m streamlit run ai/Main.py
```

4.3 使用流程

1. 浏览器打开 <http://localhost:8501>
2. 点击上传 PDF 文件
3. 等待系统加载（首次需下载 all-mpnet-base-v2 模型约 1GB）
4. 输入问题，按回车
5. DeepSeek R1 基于 PDF 内容生成回答

5. 注意事项

国内网络

代码预设 HF_ENDPOINT=https://hf-mirror.com 绕过 HuggingFace 封锁

首次启动慢

需下载 all-mpnet-base-v2 (~1GB), 后续有缓存

多 Python 环境

本机有 hermes-venv / Local Program / Roaming 三个 Python。
确保 streamlit 和包安装在同一个 Python 下

临时文件

每上传一个 PDF 用 tempfile 自动清理, 支持并发

Ollama 地址

默认 http://localhost:11434, 如需改地址传 Ollama(base_url=...)

文件路径

解决后先改回 python 路径再 pip install 和 streamlit run 命令

langchain 版本

必须用 1.2.10。新版 (>=1.3) 移除了 langchain.prompts 和 langchain.chains